Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики
Кафедра системного программирования ИСП РАН
Лаборатория (laboratory name)

Выпускная квалификационная работа бакалавра

Разработка системы логгирования из гостевой системы напрямую в файл на хосте на основе MPSC кольцевого буфера

Автор:

Студент Б05-232 группы

Абубакиров Владимир Андреевич

Научный руководитель:

Кандидат физико-математических наук

Камкин Александр Сергеевич

Научный консультант:

научная степень

Скородумов Дмитрий Павлович

Москва 2026

Аннотация

Разработка системы логгирования из гостевой системы напрямую в файл на хосте на основе MPSC кольцевого буфера

Абубакиров Владимир Андреевич

Современные платформы виртуализации реализуют сложный набор подсистем, обеспечивающих взаимодействие между гостевыми и хостовыми операционными системами. Однако для задач низкоуровневой отладки, профилирования и формирования корректных отчётов об ошибках критически важным является наличие высокоэффективного канала передачи диагностических данных из гостевой операционной системы на хост. В рамках работы был разработан подобный механизм, основанный на lock-free кольцевом буфере в общей памяти.

Упомянуть про сравнение с virtio-serial / virtio-console

Содержание

1 Введение **4**

2 Постановка задачи **8**

3 Обзор существующих решений **9**

4 Исследование и построение решения задачи **10**

5 Описание практической части **11**

6 Заключение **12**

1 Введение

Развитие информационной инфраструктуры, начиная от облачных платформ и центров обработки данных, заканчивая персональными компьютерами, всё чаще полагается на технологии виртуализации.

Виртуализация — группа технологий, основанных на преобразовании формата или параметров программных или сетевых запросов к компьютерным ресурсам с целью обеспечения независимости процессов обработки информации от программной или аппаратной платформы информационной системы. [1]

Простые пользователи используют т.н. виртуальные машины для запуска альтернативных операционных систем на своих рабочих станциях для тестирования подозрительных программ, для разработки и обучения. Облачные провайдеры используют виртуальные машины для обеспечения работы серверных платформ: в частности, для предоставления пользователям возможности аренды вычислительных ресурсов. Последним это позволяет не покупать собственное оборудование для разворачивания инфраструктуры, масштабировать мощности под нагрузку и платить только за фактическое использование ресурсов.

Виртуальная машина — виртуальная вычислительная система, которая состоит из виртуальных устройств обработки, хранения и передачи данных и которая дополнительно может содержать программное обеспечение и пользовательские данные. [2]

Отдельным фактором, влияющим на рост спроса на технологии виртуализации, является развитие машинного обучения и искусственного интеллекта. Развертывание и обучение моделей требует больших вычислительных ресурсов (мощных GPU и NPU). Облачные провайдеры предоставляют виртуальные машины с таким оборудованием, позволяя бизнесу и частным поль-

[1] ГОСТ Р 56938-2016. Национальный стандарт Российской Федерации. Защита информации. Защита информации при использовании технологий виртуализации. Общие положения

[2] ГОСТ Р 56938-2016. Национальный стандарт Российской Федерации. Защита информации. Защита информации при использовании технологий виртуализации. Общие положения

зователям проводить исследования, не приобретая дорогостоящее оборудование. Это дополнительно увеличивает потребность эффективного использования аппаратного обеспечения, что также достигается при помощи технологий виртуализации.

Развитие технологий виртуализации предъявляет повышенные требования к инструментам диагностики и отладки, которые должны обеспечивать сбор данных из гостевых операционных систем с минимальным влиянием на их временные характеристики. В задачах логгирования и профилирования ключевыми являются низкая задержка и высокая пропускная способность передачи диагностической информации. В связи с этим возникает необходимость создания специализированного канала связи «гость – хост», оптимизированного для однонаправленной потоковой передачи данных.

Ситуация усугубляется тем, что на одной физической рабочей станции может одновременно функционировать несколько виртуальных машин. Каждая из них генерирует свои собственные логи, операции записи которых конкурируют за общие аппаратные ресурсы. Таким образом, механизм сбора диагностических данных должен работать с минимальной задержкой не только для отдельно взятой виртуальной машины, но и масштабироваться при одновременной работе множества экземпляров.

Ещё одной особенностью современных виртуальных машин является наличие сопутствующего программного обеспечения, позволяющего пользователям бесшовно с ними взаимодействовать. Например, инструменты, обеспечивающие общий буфер обмена, перенаправление USB-устройств, синхронизацию времени и разрешения экрана. Эти системы в процессе своей работы также генерируют диагностические события, которые могут быть полезны для выявления ошибок у пользователей. Таким образом, система сбора диагностической информации должна эффективно работать с учётом функционирования нескольких самостоятельных компонентов внутри виртуальной машины.

Разработка подобной системы передачи диагностических данных под-

разумевает создание драйвера в гостевой операционной системе, который взаимодействует со специальным виртуальным устройством на стороне эмулятора. Данное устройство выступает в роле конечного приёмника сообщений, обеспечивая их сбор и последующую печать в файл. Такой подход позволяет сохранить прозрачность для пользователя, так как гостевые операционные системы воспринимают устройство как стандартное аппаратное обеспечение.

Основой данного механизма является общая память, разделяемая между гостевой операционной системой и гипервизором. Такой подход позволяет организовать канал передачи данных без применения медленных операций ввода-вывода и без дополнительных копирований между буферами.

Сложность разработки заключается в том, что в угоду минимальных задержек, синхронизация доступа к буферу осуществляется без блокировок, благодаря использованию атомарных операций, что исключает риски взаимных блокировок и обеспечивает более детерминированное время выполнения операций. Такой подход позволяет достичь высокой пропускной способности и минимальной задержки при операциях записи, что особенно важно для работы с несколькими виртуальными машинами.

Немаловажной задачей также является реализовать механизм передачи сообщений из среды BIOS (UEFI). На этом этапе происходит первичная инициализация аппаратный компонентов и сбои приводят невозможности дальнейшей загрузки системы. Отсутствие средств сбора отладочной информации существенно затрудняет выявление причин отказов.

BIOS представляет собой интерфейс между программным обеспечением и аппаратными средствами, который позволяет им общаться и взаимодействовать друг с другом. [3]

UEFI (от англ. Unified Extensible Firmware Interface, единый расширяемый микропрограммный интерфейс) был разработан консорциумом UEFI Forum на основе спецификаций, изначально предоставленных корпорацией Intel как преемника BIOS (хотя большинство образов микропрограммного обеспечения UEFI предоставляют поддержку для устаревших сервисов BIOS).

[3] Вонг А. Справочник по параметрам BIOS. — 2005

UEFI обеспечивает поддержку современного оборудования, модульной архитектуры, не зависящей от архитектуры ЦПУ и драйверов, загрузочной сети и безопасной загрузки. [4]

[4] Энциклопедия «Касперского» — UEFI

2 Постановка задачи

Необходимо формально изложить суть задачи в данной секции, предоставив такие ясные и точные описания, которые позволят в последующем оценить, насколько разработанное решение соответствует поставленной задаче. Текст главы должен следовать структуре технического задания, включая как описание самой задачи, так и набор требований к ее решению.

3 Обзор существующих решений

Здесь надо рассмотреть все существующие решения поставленной задачи, но не просто пересказать, в чем там дело, а оценить степень их соответствия тем ограничениям, которые были сформулированы в постановке задачи.

4 Исследование и построение решения задачи

Требуется разбить большую задачу, описанную в постановке, на более мелкие подзадачи. Процесс декомпозиции следует продолжать до тех пор, пока подзадачи не станут достаточно простыми для решения непосредственно. Это может быть достигнуто, например, путем проведения эксперимента, доказательства теоремы или поиска готового решения.

5 Описание практической части

Если в рамках работы писался какой-то код, здесь должно быть его описание: выбранный язык и библиотеки и мотивы выбора, архитектура, схема функционирования, теоретическая сложность алгоритма, характеристики функционирования (скорость/память).

6 Заключение

Здесь надо перечислить все результаты, полученные в ходе работы. Из текста должно быть понятно, в какой мере решена поставленная задача.

Список литературы

[1] *Mott-Smith, H.* The theory of collectors in gaseous discharges / H. Mott-Smith, I. Langmuir // *Phys. Rev.* — 1926. — Vol. 28.

[2] *Морз, Р.* Бесстолкновительный PIC-метод / Р. Морз // Вычислительные методы в физике плазмы / Ed. by Б. Олдера, С. Фернбаха, М. Ротенберга. — М.: Мир, 1974.

[3] *Киселёв, А. А.* Численное моделирование захвата ионов бесстолкновительной плазмы электрическим полем поглощающей сферы / А. А. Киселёв, Долгоносов М. С., Красовский В. Л. // Девятая ежегодная конференция «Физика плазмы в Солнечной системе». — 2014.